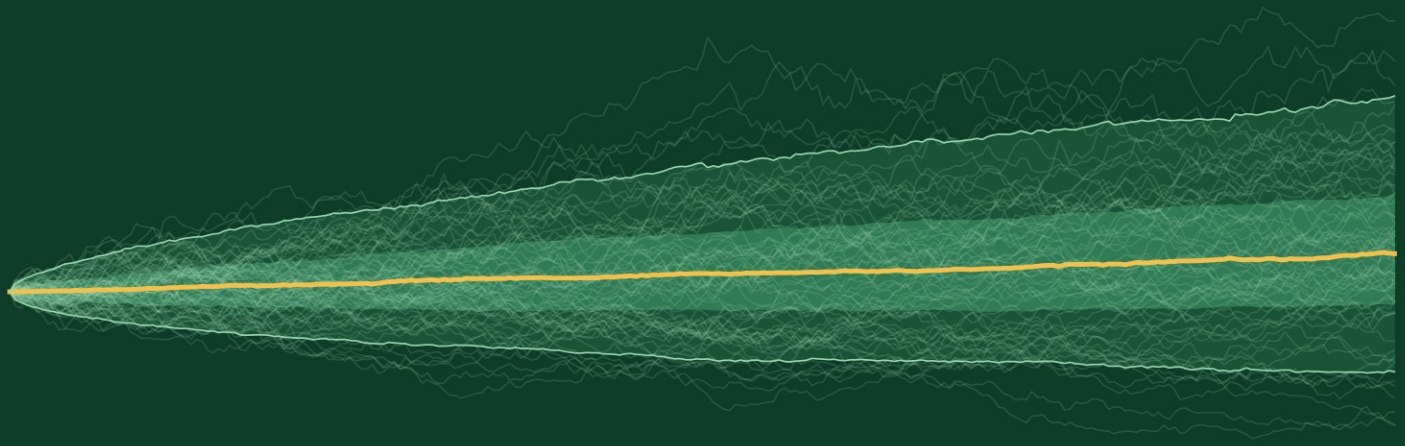
2,000 paths · 252 sessions · seed 42



FOREIGN TRADE UNIVERSITY
FACULTY OF BANKING AND FINANCE

PERSONAL PROJECT REPORT

Finfolio 2.0

Portfolio Attribution & Risk Analytics

STUDENT	Lê Bảo An
STUDENT ID	2412380002
INDIVIDUAL ROLE	Team Representative / Lead Developer
GROUP	G09 - K63
COURSE	Information Technology Applications in Finance and Banking (NHA408E)
INSTRUCTOR	Phan Trần Trung Dũng

HANOI · OCTOBER 2026

Abstract .

This report documents what I personally built in Finfolio 2.0, a multi-asset portfolio analysis application for self-directed retail investors who need to see where return and risk originate before considering a change in allocation. As Team Representative and Lead Developer, I turned the team's approved specifications into a working, tested and deployed system. I designed the application architecture and repository; wrote the numerical computation layer; implemented the constrained optimization engine; built the complete Monte Carlo simulation workflow, including seeded sampling, percentile aggregation, tail-risk measures and Web Worker execution; integrated the Python backend and data routes; wrote the automated test suites; and corrected and configured production deployment. Every claim is tied to repository files that can be run and checked. The report also states the limits of what I built: the outputs rely on historical end-of-day observations, free upstream data services and in-sample estimation, so they are analytical evidence, not forecasts or investment recommendations.

50	35	7	2,000
JAVASCRIPT TESTS	PYTHON TESTS	PRODUCT ROUTES	SIMULATED PATHS

KEYWORDS portfolio analytics · software architecture · constrained optimization · Monte Carlo simulation · Web Worker · reproducibility

Contents .

I WHAT I WAS RESPONSIBLE FOR....3

II WHAT I BUILT....4

2.1 Modular financial computation architecture....4

2.2 Constrained optimization engine....5

2.3 Joint moving-block bootstrap Monte Carlo simulation....6

III HOW I DELIVERED IT....7

3.1 Repository foundation and integration contracts....7

3.2 Verification, CI/CD, and production delivery....7

IV EVIDENCE: HOW TO CHECK MY WORK....8

V LIMITATIONS AND LESSONS LEARNED....9

VI CONCLUSION....9

— INDIVIDUAL DECLARATION....10

List of figures

Figure 1. Finfolio 2.0 system architecture: the components I built and integrated....4

Figure 2. Constrained allocation interface for minimum-volatility and approximate maximum-Sharpe analysis....5

Figure 3. Monte Carlo fan chart, terminal distribution, and disclosed tail-risk outputs....7

I WHAT I WAS RESPONSIBLE FOR

Finfolio 2.0 was designed to address a practical interpretation problem. A retail investor may know the market value of a portfolio and its overall gain or loss, yet still be unable to explain which holdings generated the result, where risk is concentrated, or how a constrained alternative allocation would have behaved over the same historical observations. The product therefore had to connect several layers that are often treated separately: heterogeneous market data, currency conversion, portfolio mathematics, interactive visualization, and language that communicates uncertainty responsibly. The technical challenge was not limited to implementing isolated formulas. It required a stable architecture in which each layer could be inspected, tested, and replaced without obscuring the meaning of the final output.

As Team Representative and Lead Developer, I was accountable for converting the approved specifications into a working, testable, and deployable application. I structured the repository, established the interfaces between Python data adapters and browser-side JavaScript modules, implemented the core computation, and integrated the results into the product routes. I also owned the constrained optimization engine and the complete Monte Carlo implementation, including seeded random sampling, percentile aggregation, tail-risk measures, Web Worker execution, and the simulation interface data contract. These responsibilities are evidenced by *demo/finance.mjs*, *demo/optimizer.mjs*, *demo/simulation.mjs*, *demo/research-worker.mjs*, the Python data modules, the test suites, and the production configuration.

My contribution at a glance

Architecture and repository	Organized the repository into application, documentation, data, test and deployment areas; defined the interfaces between Python data adapters and JavaScript modules.
Computation layer	Wrote finance.mjs, attribution.mjs and metrics.mjs: value paths, contributions, covariance, volatility, Euler component risk, FX attribution.
Optimization	Wrote optimizer.mjs: constrained minimum volatility and approximate maximum Sharpe, with feasibility checks.
Monte Carlo	Wrote simulation.mjs and research-worker.mjs: joint moving-block bootstrap, seeded sampling, percentiles, VaR and CVaR, Web Worker execution.
Backend integration	Built canonical data representations and the routes for market data, symbol search, account configuration and data status.
Verification and delivery	Wrote the JavaScript and Python test suites (50 + 35 tests); configured Vercel and repaired the data-refresh publication logic.

WHERE MY WORK BEGINS AND ENDS

Financial definitions, data-provenance requirements, acceptance cases and the visual system were specified by my teammates. My job was to implement and integrate those specifications without silently changing their meaning.

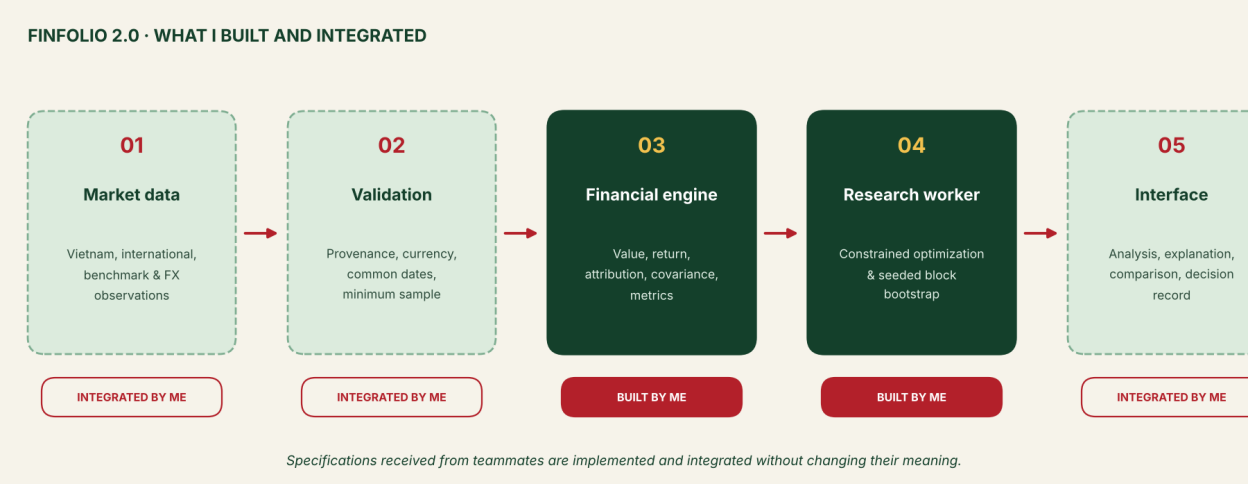


Figure 1. Finfolio 2.0 system architecture: the components I built and integrated

Source: Author's diagram based on the Finfolio 2.0 repository.

II WHAT I BUILT

2.1 Modular financial computation architecture

I implemented the numerical layer as JavaScript ES modules rather than embedding it in interface event handlers. In *finance.mjs* I built portfolio value paths, holding weights, return contributions, covariance estimates, annualized volatility and Euler component risk. In *attribution.mjs* I implemented the decomposition of foreign-currency returns, and in *metrics.mjs* the supplementary risk and relative-performance statistics disclosed in the methods documentation. Because the interface and the automated tests call the same functions, a displayed number cannot use logic different from the verified numerical procedure.

I also imposed explicit data contracts. Holdings, price matrices, exchange-rate series and benchmark observations arrive as separate objects with date and currency metadata, and before any relative or risk calculation my code checks that the series contain enough common observations and that invalid values have been removed. The calculation layer therefore depends on declared inputs rather than on provider-specific response shapes, and it picks up cleanly from the data requirements handed to me.

The portfolio value path is based on an initial allocation and buy-and-hold quantities. In simplified form, the value at date t is represented as follows:

$$V_t = \Sigma_i \; q_i \cdot P_{i,t} \cdot FX_{i,t} \qquad (1)$$

This construction allows the chart to display actual portfolio value in VND rather than a normalized base-100 index. It also creates a common numerical foundation for period return, contribution, benchmark comparison, optimization comparison, and simulation starting value.

2.2 Constrained optimization engine

I implemented two reference-allocation workflows in *optimizer.mjs*: constrained minimum volatility and approximate maximum Sharpe. Both use a long-only feasible set, require weights to sum to 100%, and apply limits to each instrument and to the commodity-proxy group. The implementation first evaluates whether the constraints are feasible. For example, an allocation cannot be formed when the sum of permitted maximum weights is below 100%. Detecting this condition before numerical search prevents the system from presenting a plausible-looking vector that violates the user's stated limits.

For minimum volatility I used a conditional-gradient approach, which suits linear constraints. For maximum Sharpe I evaluate multiple starting points, namely equal-weight, inverse-volatility and minimum-volatility candidates, because the ratio objective is more sensitive to estimated means and initialization. My implementation returns convergence information, and the application labels the result as an estimated reference allocation. When the iteration limit is reached, it discloses that the numerical threshold was not attained rather than describing the candidate as an exact global optimum.

$$\textit{Minimum volatility:}\quad \textit{minimize}\ \ w^T \Sigma\, w \tag{2}$$

$$\textit{Maximum Sharpe:}\quad \textit{maximize}\ \ (w^T \mu - r^R) \,/\, \sqrt{(w^T \Sigma\, w)} \tag{3}$$

The before-and-after comparison I implemented uses the same historical observations and assumptions for both allocations, which isolates the effect of weights from changes in sample period or data availability. Figure 2 shows the screen through which the user selects the objective and reviews both the estimated metrics and the proposed weights.

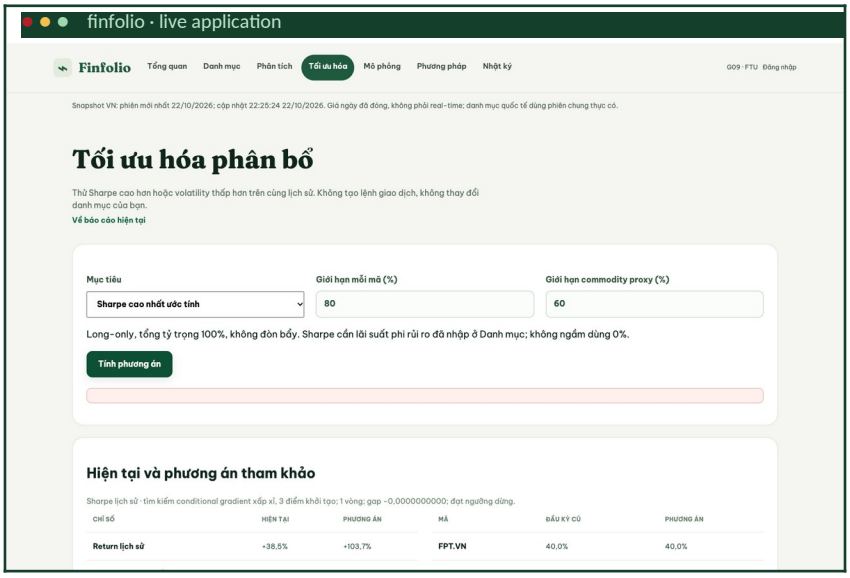


Figure 2. Constrained allocation interface for minimum-volatility and approximate maximum-Sharpe analysis

Source: Finfolio 2.0 application, project screenshot.

2.3 Joint moving-block bootstrap Monte Carlo simulation

The Monte Carlo workflow represents my most distinctive individual technical contribution. A conventional simulation based on independent normal draws would suppress several characteristics visible in financial returns, including fat tails, short volatility clusters, and contemporaneous dependence between holdings. I therefore implemented a joint moving-block bootstrap. The algorithm samples common blocks of five consecutive historical return observations across all assets. Sampling the same dates for every asset preserves their cross-sectional relationship, while retaining short sequences preserves part of the observed temporal dependence.

I configured the simulation with 2,000 paths over a 252-session horizon and used a seeded linear congruential generator, so running the same portfolio, history, horizon, path count and seed produces the same percentile bands and tail statistics.

2,000	252	5	42
PATHS	SESSION HORIZON	DAY BLOCK LENGTH	REPRODUCIBLE SEED

Each simulated path begins at the actual final portfolio value and evolves using the sampled multivariate returns. My aggregation stage reports the 5th, 25th, 50th, 75th and 95th percentiles, the proportion of terminal values below starting capital, simulated VaR and CVaR, and a terminal-value histogram. These values describe outcomes inside the resampling model; the interface explicitly avoids presenting them as verified future probabilities.

I run the computation through *research-worker.mjs*. Executing thousands of paths and repeated matrix operations on the browser's main thread would temporarily block pointer, keyboard and rendering events. The Web Worker receives a structured message, performs the numerical loop outside the interface thread, and returns serializable results through *postMessage*. This kept the interface responsive and gave optimization and simulation one execution path, without allowing UI state to alter the numerical method.

Figure 3 presents the resulting percentile fan, terminal distribution, and disclosed tail-risk outputs in the application.

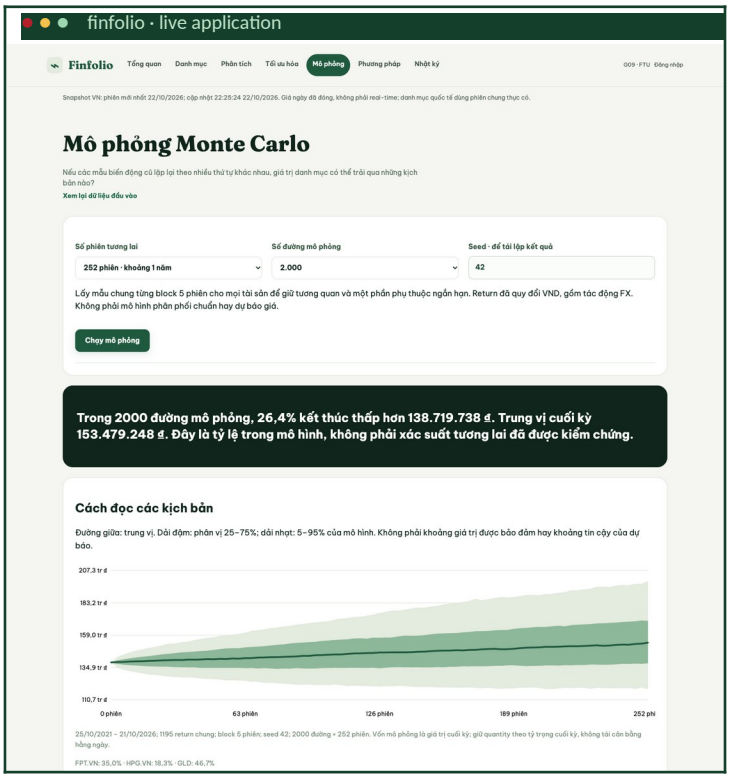


Figure 3. **Monte Carlo fan chart, terminal distribution, and disclosed tail-risk outputs**

Source: Finfolio 2.0 application, project screenshot.

III HOW I DELIVERED IT

3.1 Repository foundation and integration contracts

During the first project stages, I organized the repository into application, documentation, data, test, and deployment areas. The browser application uses lightweight JavaScript modules and native SVG rendering, while Python modules provide provider-specific data retrieval and serverless API routes. This structure was selected because the project required transparent numerical logic and direct deployment rather than a large framework. It also allowed the team to inspect individual modules during verification and to trace a displayed result back through the calculation and data layers.

By Week 3, the central task was to stabilize interfaces between workstreams. I implemented canonical representations for holdings, time series, exchange rates, and benchmark data; created routes for market data, symbol search, account configuration, and data-status metadata; and preserved provider and freshness information. The data-status endpoint deliberately excludes the user's holdings. Its purpose is to state the latest available market session and snapshot time, allowing the application to determine whether data may be stale without exposing portfolio content.

3.2 Verification, CI/CD, and production delivery

I protected the implementation with JavaScript and Python test suites. The final audit recorded 50 JavaScript tests and 35 Python tests passing on 23 October 2026. They cover numerical identities, constraint feasibility, deterministic simulation, freshness behavior, symbol search, classification and API boundaries. They do not prove universal financial accuracy or usability; they provide reproducible evidence that defined invariants remain stable when code changes.

Deployment required coordination between GitHub Actions and Vercel. The scheduled data workflow retrieves and validates closed daily observations before publishing a dated snapshot. The Vercel configuration maps the static application and serverless routes to one public project. When the earlier refresh job attempted to publish to a missing branch, I corrected the publication logic and verified the workflow against a complete Vietnam universe snapshot. The final release also exposed source timestamps inside the application so a successful deployment could not be mistaken for current market data when the underlying session was old.

IV EVIDENCE: HOW TO CHECK MY WORK

The repository evidence provides a reproducible chain from specification to output. The most direct numerical checks are *demo/finance.test.mjs* for portfolio and Euler calculations, *demo/research.test.mjs* for optimization and simulation, and the Python unit tests for provider and API behavior. A reviewer can run *npm test* and *python3 -m unittest discover -s demo -p 'test_*.py'*, then open the corresponding routes to compare the machine-checked invariants with the visible product behavior.

WHAT I BUILT	REPOSITORY EVIDENCE	VERIFICATION PROCEDURE
Financial computation	demo/finance.mjs demo/attribution.mjs demo/metrics.mjs	Run the finance and attribution tests; reconcile return and Euler component totals.
Optimization	demo/optimizer.mjs demo/research.test.mjs	Compare minimum-volatility and maximum-Sharpe results under feasible caps.
Monte Carlo	demo/simulation.mjs demo/research-worker.mjs	Repeat seed 42 and confirm identical percentiles, VaR, and CVaR.
Data and API integration	demo/server.py demo/vn_data.py demo/global_data.py	Start the local server and inspect market-data and data-status responses.
Deployment	vercel.json .github/workflows/update-vn-data.yml	Inspect the production routes and the corresponding workflow run.

The final outcome was a seven-route application covering portfolio entry, analysis, optimization, simulation, methodology, and private decision recording. The main analytical flow processes actual instrument values and currencies, converts observations to a common VND basis, and presents contribution and risk evidence

alongside benchmark comparison. Optimization and simulation are accessible as optional analytical extensions, which prevents them from obscuring the core attribution task.

V LIMITATIONS AND LESSONS LEARNED

The system operates on end-of-day observations rather than intraday data, and its international and Vietnamese series may close at different times. Free providers introduce rate limits and possible historical revisions. The current calculation assumes buy-and-hold quantities, excludes transaction costs and taxes, and does not fully model dividend reinvestment. The maximum-Sharpe procedure and covariance estimates remain in-sample, while the bootstrap can only resample regimes represented in the historical record. These conditions constrain interpretation even when the software executes correctly.

The main technical lesson was that numerical correctness requires an observable contract. A formula needs admissible inputs, a testable invariant, a defined unit, and an explanation of what the output permits the user to conclude. Modular computation and seeded simulation made the system reproducible; explicit convergence and freshness disclosures made its uncertainty visible. Separating background computation from the interface also demonstrated that performance is part of analytical reliability: an unresponsive page can prevent a user from inspecting or challenging the result.

> **THE MAIN LESSON**
>
> *Numerical correctness requires an observable contract: admissible inputs, a testable invariant, a defined unit, and an explanation of what the output permits the user to conclude.*

VI CONCLUSION

My contribution connected the project's methodological, data, quality, and interface specifications into an operational system. I implemented the core financial engine, constrained optimization, the complete Monte Carlo workflow, Web Worker execution, backend integration, automated verification, and production configuration. The resulting artifacts can be inspected independently and reproduced through the repository tests and application routes.

The work also clarified the boundary of a Lead Developer's accountability in a financial application. I was responsible for faithful and reproducible implementation, while the meaning of financial measures, the admissibility of data, the acceptance conditions, and the visual hierarchy remained governed by the respective team owners. This division enabled Finfolio 2.0 to present complex analysis without treating software output as financial advice.

INDIVIDUAL DECLARATION

DECLARATION

I confirm that this report reflects the work that I am prepared to explain, calculate, demonstrate, or reproduce in Finfolio 2.0. Shared project artifacts are identified as collaborative where applicable, and my individual accountability is defined by the evidence presented in this report.

FULL NAME	Lê Bảo An
STUDENT ID	2412380002
SIGNATURE	
DATE	